

Architecture Summary: CrewAI E-commerce Triage Pipeline

This document summarizes the architecture implemented in the attached notebook, which builds a CrewAI-based e-commerce customer-support triage workflow using four specialized agents in sequence.

Overview

The notebook implements a sequential support pipeline that accepts a customer complaint plus mock order data, then transforms that input into a structured decision trail covering issue identification, policy interpretation, resolution recommendation, and final escalation review. The design is intentionally modular so that each agent performs one bounded responsibility and passes structured context to the next stage.

Agent roles

Agent	Responsibility	Main output
Order Issue Identification Agent	Extracts facts, classifies the complaint, estimates urgency, and assigns confidence.	Structured issue case with issue type, urgency, confidence, and missing information flags.
Policy Interpretation Agent	Applies return, refund, replacement, and exception rules to the structured case.	Policy decision including eligibility, allowed actions, and policy exceptions.
Resolution Recommendation Agent	Chooses the best next action that is consistent with policy and customer context.	Resolution plan such as refund, replacement, wait-and-watch, or reject with explanation.
Escalation Agent	Determines whether a human team must review the case based on risk or ambiguity.	Escalation decision with routing rationale and suggested next step.

Task flow

The flow is linear. First, the triage agent reads the raw complaint and order details to build a structured issue representation. Second, the policy agent evaluates that issue against a static policy text embedded in the notebook. Third, the resolution agent proposes the most appropriate action

permitted by policy. Finally, the escalation agent checks risk triggers and decides whether the case can be auto-resolved or should be sent to a human queue.

This architecture reduces duplicated reasoning because downstream agents consume normalized outputs instead of reinterpreting the original complaint from scratch. It also makes debugging easier because each stage can be inspected independently through the notebook's verbose agent execution style.

Escalation logic

The notebook defines explicit escalation rules rather than relying only on free-form model judgment. The configured rules escalate cases when the order value exceeds 500, when the order ID is missing, when classification confidence is low, when a delivered-but-not-received claim has stronger risk signals, when a repeat refund pattern appears, or when an exception case involves non-returnable, urgent, regulated, or perishable items.

This means the escalation agent functions as a guardrail layer rather than a generic summarizer. In practice, it protects the workflow from automatically resolving risky, ambiguous, or potentially fraudulent cases.

Sample inputs and outputs

The notebook includes example scenarios to show how the pipeline behaves across common support situations. One scenario covers a damaged high-value electronics order, another covers delayed shipment handling, and another tests a return request that falls outside the policy window.

In the damaged high-value case, the triage stage identifies a damage-related complaint with strong urgency and confidence, the policy stage marks the case as eligible for refund or replacement because damaged items reported within seven days qualify, the resolution stage recommends a next action such as replacement, and the escalation stage still routes the case for human review because the order value exceeds the threshold. In an outside-window return example, the policy stage limits available actions, which demonstrates that the resolution agent is constrained by policy rather than optimizing only for customer satisfaction.

Design rationale

Several design choices stand out in the notebook. The first is separation of concerns: each agent has a narrow role, which keeps prompts simpler and makes behavior easier to maintain over time. The second is deterministic model configuration: the shared LLM is initialized with `gpt-4o-mini` and `temperature=0`, which supports repeatable outputs for operational workflows where consistency matters more than creativity.

The third is the use of structured intermediate outputs. This is important because structured JSON-like results are easier to validate, log, and integrate with downstream systems such as helpdesk tools, audit reports, or dashboards. The fourth is explicit policy text embedded in the notebook, which makes the business rules transparent and editable without redesigning the entire agent chain.

Architecture assessment

From an architecture perspective, the notebook demonstrates a practical pattern for agentic orchestration in customer operations. It is strongest where business rules are stable, decisions benefit from staged reasoning, and escalation criteria must be auditable.

The main trade-off is that static policy text can become a maintenance burden if policy changes frequently or must vary by geography, category, or merchant. A natural next step would be externalizing policy and escalation rules into configuration or retrieval-backed documents while preserving the same four-stage control flow.

Output - 1

Step 6 - Run the Triage Pipeline

Provide a plain-text patient description and call `kickoff()`. The pipeline injects `user_query` into all four tasks and runs them in sequence. Change the text in `user_query` to any of the test cases at the bottom to see how the system responds to different severity levels.

[21]

```
1 # Sample user query - a critical, high-severity case with no manual intervention needed
2 # This should trigger 'Escalation agent result
3 user_query = """
4 Customer issue: My headphones arrived cracked today and I want a refund or replacement.
5 Order ID: ORD-10452
6 Order value: 1299
7 Delivery status: Delivered today
8 Days since delivery: 0
9 Item category: Electronics
10 Previous refunds on account: 0
11 """
12
13 # kickoff() starts the sequential pipeline and injects the input into all tasks
14 result = triage_pipeline.kickoff(inputs={"user_query": user_query})
15
16 print("\n" + "=" * 60)
17 print("FINAL TRIAGE OUTPUT")
18 print("=" * 60)
19 print(result)
```

Output

```
}
```

> Finished chain.

```
[DEBUG]: == [Escalation Agent] Task output: {
  "EscalationDecision": {
    "escalate": true,
    "rationale": "High urgency and high order value. The system has already decided to escalate the case due to these factors. The
```

```
proposed solution of a replacement is appropriate, considering the high urgency and high order value.",
    "manualReviewRequired": false,
    "nextSteps": ["Initiate replacement process as per proposed ResolutionPlan, ensuring delivery within the next 3-5 business days."]
  }
}
```

=====

FINAL TRIAGE OUTPUT

=====

```
{
  "EscalationDecision": {
    "escalate": true,
    "rationale": "High urgency and high order value. The system has already decided to escalate the case due to these factors. The
proposed solution of a replacement is appropriate, considering the high urgency and high order value.",
    "manualReviewRequired": false,
    "nextSteps": ["Initiate replacement process as per proposed ResolutionPlan, ensuring delivery within the next 3-5 business days."]
  }
}
```

Output - 2

```
1 # Sample user query - Outside return period item - request to be denied
2 # This should trigger 'Escalation agent result'
3
4 user_query = """
5 Return outside policy window.
6 I changed my mind and want to return the beauty product I received more than a month ago.
7 Order ID: ORD-88421
8 Order value: 349
9 Delivery status: Delivered 38 days ago
10 Days since delivery: 38
11 Item category: Beauty
12 Previous refunds on account: 0
13 """
14 # kickoff() starts the sequential pipeline and injects the input into all tasks
15 result = triage_pipeline.kickoff(inputs={"user_query": user_query})
16
17 print("\n" + "=" * 60)
18 print("FINAL TRIAGE OUTPUT")
19 print("=" * 60)
20 print(result)
```

^ Output

```
> Finished chain.
[DEBUG]: == [Escalation Agent] Task output: ``json
{
  "needs_escalation": false,
  "reasons": [],
  "priority": "low",
```

```
    "recommended_team": null,
    "notes": "The order is beyond the eligible return period and there are no exception flags to override this decision. As such, no
escalation is required."
}
...

=====
FINAL TRIAGE OUTPUT
=====
``json
{
  "needs_escalation": false,
  "reasons": [],
  "priority": "low",
  "recommended_team": null,
  "notes": "The order is beyond the eligible return period and there are no exception flags to override this decision. As such, no
escalation is required."
}
...
```